

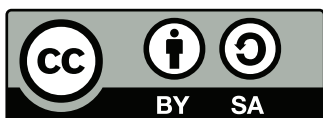


Universidad Complutense de Madrid

Documentación del software desarrollado para la práctica de control de un motor de cc

Juan Jiménez

8 de mayo de 2026



El contenido de estos apuntes está bajo licencia Creative Commons Attribution-ShareAlike 4.0
<http://creativecommons.org/licenses/by-sa/4.0/>
©Juan Jiménez

Índice general

1. Primeros pasos	2
1.1. Software necesario	2
2. El entorno de desarrollo de Visual Studio Code (VSC)	4
2.1. Instalación de las extensiones STM32Cube	5
2.2. Instalación de las extensiones para Python	5
3. Configuración de la placa con STM32CubeMX	6
3.1. Placa en modo debug y reseteo del pin PB3	6
3.2. Configuración del Reloj.	6
3.3. Configuración de un puerto serie (UART), a través del puerto USB de la placa.	7
3.4. Configuración del resto de pines para el control del motor	8
3.5. Habilitar un timer para que lea periódicamente el número de pasos (Encoders) que ha avanzado el motor.	9
3.6. Instalar y activar CMSIS-DSP	10
3.7. Definir CMake para la compilación y el enlazado y obtener el esqueleto del código del proyecto	10
4. Programación del Microcontrolador en C	11
4.1. Estructura del proyecto	11
4.2. main.h y main.c	12
4.3. Configuración de Periféricos (STM32CubeMX)	12

Índice de figuras

2.1. Ventana de Visual Studio Code	5
3.1. Ventanas de inicio y selección de placa	7
3.2. Vista de la pestaña de configuración del reloj, con el reloj configurado a 100MHz.	8

Índice de Tablas

3.1. Mapa de conexiones nativas entre el sistema electromecánico y la placa STM32 Nucleo.	8
---	---

Resumen

Estas notas son complementarias al guión de la práctica y explican todo el proceso de construcción del software empleado en la práctica. Dan información de los IDEs empleados para el desarrollo del software, de su instalación, de la configuración de la placa de control empleada y de la estructura de los programas desarrollados

Capítulo 1

Primeros pasos

1.1. Software necesario

En primer lugar, como se ha explicado ya en el guión de la práctica, vamos a emplear la placa STM32F411E, de la compañía ST, para controlar el motor. En la jerga de los programadores de microcontroladores, esta placa es conocida como la *Black Pill*. Una de sus ventajas, es que tiene un excelente soporte de software y herramientas de desarrollo.

En esta sección vamos a describir como a asumir que estamos trabajando con un ordenador personal con sistema operativo Linux, en concreto vamos a emplear Ubuntu 24.04.4 LTS.

<https://ubuntu.com/download/desktop>

Algunas herramientas básicas necesarias son:

1. Visual studio Code (VSC). Será el entorno de desarrollo que emplearemos para escribir, compilar y descargar el código que utilizará la placa ST. También servirá para desarrollar el código en Python que nos permitirá comunicarnos y controlar la placa desde el ordenador. Para instalarlo en Ubuntu, lo ideal es descargar directamente el paquete .deb, de la página web de VSC,

https://code.visualstudio.com/thank-you?dv=linux64_deb

para instalarlo ir a la línea de comandos:

```
sudo apt install ./Descargas/code_*.deb
```

Ver descripción detallada en el capítulo ??

2. STM32CubeMX: Es la herramienta gráfica oficial de ST. Sirve para configurar los pines, el reloj (clock) y generar el código base en C.

<https://www.st.com/en/development-tools/stm32cubemx.html>

3. STM32CubeCLT: Tiene todas las herramientas de ST, necesarias para compilar, enlazar etc. Para instalarlo Descargar de la página web de ST,

<https://www.st.com/en/development-tools/stm32cubecolt.html>

el que pone Debian Linux installer. Descomprimir,

```
unzip st-stm32cubecolt_1.21.0_27995_20260219_1804_amd64.deb_bundle.sh.zip
```

Dar Permisos

```
chmod +x st-stm32cubecolt_1.21.0_27995_20260219_1804_amd64.deb_bundle.sh
```

Lanzar el instalador

```
sudo ./st-stm32cubeclt_1.21.0_27995_20260219_1804_amd64.deb_bundle.sh
```

Presentará en el terminal la licencia de uso, ir hasta el final y aceptarla. Debería instalarlo en:

```
/opt/st/stm32cubeclt_1.21.0/
```

Es toolset STM32CubeCLT incluye:

- GNU C/C++ for Arm® toolchain tales como arm-none-abi-gcc(compilador), arm-none-abi-nm (symbol viewer) y otros.
 - GDB debugger cliente y servidor (Depurador). Depurar el programa ejecutándolo en la propia placa
 - STM32CubeProgrammer (STM32CubeProg) utility (Programador) Descargar el ejecutable desde el ordenador a la placa.
 - System view descriptor files (.SVD) for the entire STM32 MCU portfolio.
 - Map file associating STM32 MCUs and MCU development boards to the appropriate SVD
4. Cmake y Ninja: fundamentales para gestionar la construcción del proyecto: Librerías, dependencias, etc.
 5. ST-Link Drivers: permite la comunicación entre el ordenador y la placa de desarrollo a través de un puertos USB. Estas dos últimas herramientas se instalan directamente desde VSC. (Ver capítulo ??)
 6. Python y los modulos de Python: serial, struct, csv, sys, time, collections, datetime, pyqtgraph.
 7. (Opcional) Si se va a hacer el análisis de los datos y los modelos del motor en python, entonces es conveniente instalar también los módulos: pandas, numpy, matplotlib.pyplot y control

Para instalar STM32CubeMx, lo ideal es obtenerlo de la página web de ST,

<https://www.st.com/en/development-tools/stm32cubemx.html>

Igualmente, para STM32CubeProgrammer,

<https://www.st.com/en/development-tools/stm32cubeprog.html>

El resto de las herramientas, se pueden instalar directamente a través de Visualstudio Code, Mediante las extensiones para SMT32.

En el caso de Python, es bastante cómodo instalar directamente anaconda.

<https://www.anaconda.com/downloadP>

Varios de los módulos de Python necesarios vienen instalados por defecto con Anaconda y el resto son fácilmente instalables empleando **conda**.

Capítulo 2

El entorno de desarrollo de Visual Studio Code (VSC)

Entre las diferentes opciones existentes de IDEs (Integrated development Environments), se ha optado por emplear el programa Visual Studio Code para el desarrollo de este proyecto.

Si abrimos el programa, una vez instalado, accedemos a una ventana como la se muestra en la figura ??.

Vamos a describir brevemente los elementos más importantes.

- Barra de actividad: Es la barra vertical localizada a la izquierda del todo de la ventana. Esta barra permite cambiar la vista del panel situado inmediatamente a su derecha. Las distintas opciones hacen referencias a tareas distintas realizables desde VSC. Así el icono superior (Explorer nos permite navegar por los directorios que tenemos abierto, seleccionar archivos para abrirlos en el editor etc. Debajo tenemos una opción de búsqueda (Search). Un control de cambios basado en git (Source control). Un botón para ejecución y depuración de código (RUN and debug), Un botón muy útil que nos permite explorar las extensiones (Extensions) disponibles, es decir configuraciones específicas para manejar distintos tipos de lenguajes de programación, etc.
- Barra Lateral, inmediatamente a la derecha de la anterior, ofrece distintas vistas y paneles, dependiendo de la actividad seleccionada en la barra de actividad. En la figura ??, se ve el panel correspondiente a las extensiones, porque la actividad seleccionada, ha sido precisamente el botón extensions. Se pueden recorrer las extensiones disponibles, comprobar si están o no instaladas, y seleccionarla para su instalación/desinstalación, configuración, etc.
- El editor. Ocupa el area central. Su objetivo es escribir y editar código. Es posible abrir varios editores uno al lado del otro y de este modo trabajar en varios archivos simultáneamente. En la figura ??, se muestra un fichero de texto que contiene la fuente el $\text{\LaTeX}$ con la que se han elaborado estas notas. A su derecha, se ve el texto resultante en un archivo pdf.
- Barra de estado (Status Bar) Situada en la parte inferior de la ventana, ofrece información contextual relacionada con el estado actual de VSC y la actividad que se está llevando a cabo.
- Paleta de Comandos (Command Palette) Se accede pulsando los comandos `Ctrl + Shift + P`, es muy útil porque permite el acceso rápido a muchos comandos y características de VSC, sin tener que navegar por menús de opciones.
- Paneles. Están localizados debajo del editor. Muestran distintas herramientas muy útiles. Terminales del sistema, Salidas para los distintos compiladores, Consolas de depuración, panel de problemas etc.
- Tabs. Situados en la parte superior del editor, permiten intercambiar la vista entre los archivos abiertos.

Una vez instalado VSC debemos seleccionar las extensiones adecuadas para poder compilar código C, para la placa, descargarlo y, en su caso, poder depurarlo.

Además, podemos también instalar las extensiones para escribir y ejecutar código en python. De esta manera, con un solo IDE, podemos generar todo el código necesario; tanto el código en C para la placa controladora como en código en Python para interactuar con la placa desde el ordenador.

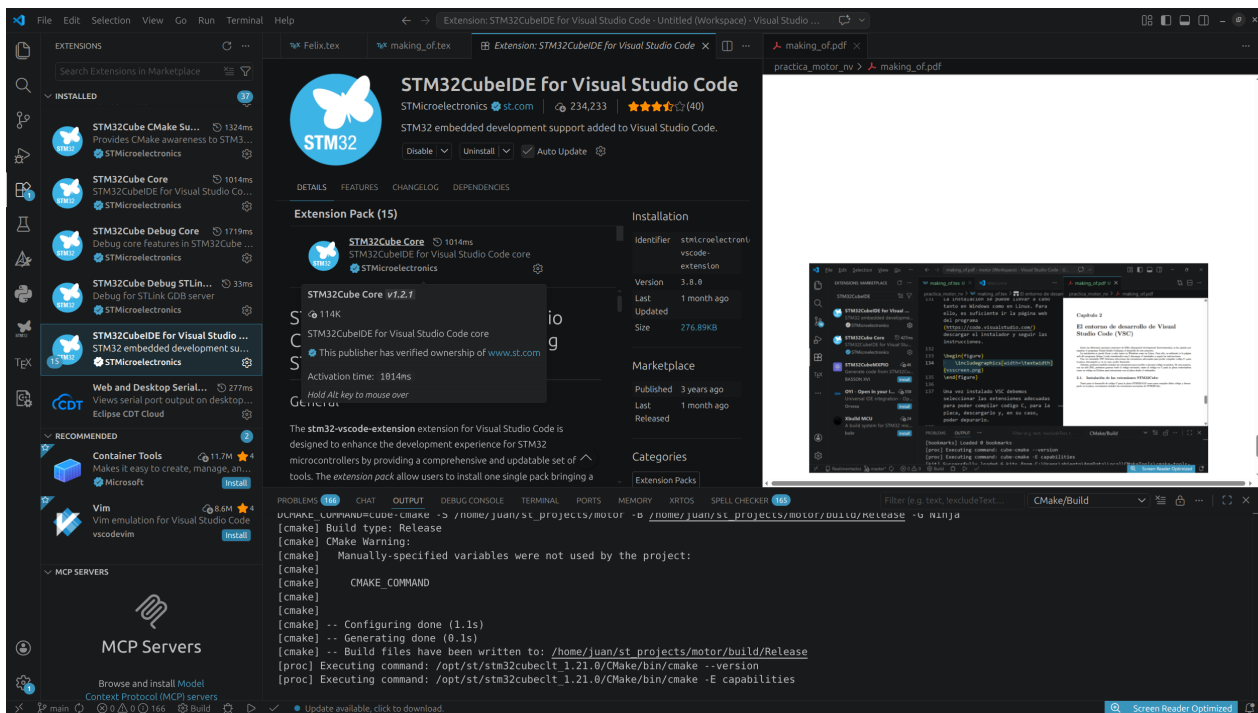


Figura 2.1: Ventana de Visual Studio Code

2.1. Instalación de las extensiones STM32Cube

Tanto para el desarrollo de código C para la placa STM32F411E como para compilar dicho código y descargarlo en la placa, necesitamos instalar las extensiones necesarias de STM32Cube.

Para hacerlo, pulsamos en la barra de actividades, el botón de las extensiones que tiene un icono con cuatro cuadros. Automáticamente aparecerá en la barra lateral las extensiones disponibles. Lo más sencillo es escribir en la barra de búsqueda la extensión que deseamos instalar: **STM32CubeIDE for Visual Studio Code**. Las extensiones de ST, empujan como símbolo una mariposa sobre un círculo azul, seleccionamos la que hemos buscado, pulsamos en el botón 'install'. Dejamos que complete la instalación. Estamos ya en condiciones de escribir, compilar y descargar código para nuestra placa ST.

2.2. Instalación de las extensiones para Python

VSC, puede emplear directamente la instalación de Python existente en nuestro ordenador. Sin embargo, como ya hemos indicado, es particularmente cómodo manejarlo desde la distribución de anaconda.

Para poder manejar python desde VSC podemos emplear la extensión 'Pylance'. El procedimiento de instalación es idéntico al que hemos visto para Las extensiones de ST. Buscamos Pylance en el buscador de las extensiones y pulsamos el botón 'Install'.

Una vez instalado, VSC reconocerá cualquier archivo con extensión .py como propio de Python y nos mostrará a, la altura de los tabs, un triángulo verde. Si lo pulsamos ejecutará, en el terminal, disponible en los paneles, el código de archivo .py abierto en el editor activo.

Es posible que en la primera ejecución nos pregunte que interprete de python queremos emplear, seleccionar el señalado como 'conda', asociado a la distribución de Anaconda.

Capítulo 3

Configuración de la placa con STM32CubeMX

STM32CubeMx, es una herramienta extremadamente útil para configurar el Hardware de nuestra placa. Conocer a fondo todas sus posibilidades está fuera de los objetivos de estas notas. Aquí nos vamos a centrar en configurar la placa para que pueda controlar nuestro motor. Lo vamos a hacer paso a paso.

Al ejecutar STM32CubeMX, se abre una primera ventana en la que se nos permite abrir proyectos ya existentes o crear proyectos nuevos figura (??). Entre las opciones para crear proyectos nuevos, podemos emplear tanto el botón **ACCESS TO MCU SELECTOR** como el botón **ACCESS TO BOARD SELECTOR**. La diferencia es que en el primer caso nos selecciona el microcontrolador, y no nos configura nada por defecto. Si seleccionamos la placa el programa sabe qué pines están conectados al led y al botón azul de usuario y además configura la emulación del puerto serie, a través del puerto USB de la placa, para comunicar la placa con el ordenador. Si empleamos la primera opción, tendremos que configurar nosotros a mano estas opciones, caso de que queramos usarlas.

Vamos a seleccionar directamente la placa núcleo que la que vamos a trabajar. Para ello pulsamos el botón **ACCESS TO BOARD SELECTOR**. Se abrirá una nueva ventana, figura (??). Seleccionamos en la entrada **comercial part number**, arriba a la izquierda, nuestra placa: **NUCLEO F411RE**. Esperamos a que la seleccione y nos abrirá una pequeña ventana de diálogo en la que nos sugiere inicializar todos los periféricos con las opciones por defecto. Seleccionamos **no**. Se cierra la ventana de selección y en la ventana principal de STM32CubeMX aparecen varias pestañas y se nos muestra el pinout de la placa con los pines que ya están configurados por defecto, el color y 'fijados' con una chincheta figura (??).

3.1. Placa en modo debug y reseteo del pin PB3

Lo primero que vamos a hacer, es poner la placa en modo debug. Hacemos esto para que nos deje programar la placa siempre que queramos sin necesidad de emplear el botón de reset. Para ello, desplegamos el menú **System Core**, seleccionamos la opción **SYS** y se nos abrirá un panel de configuración, figura (??). En la entrada que puede **Debug**, seleccionamos **Serial Wire**. A continuación vamos a liberar el pin B3. Por defecto este pin está configurado precisamente para emplear el método de depuración **Serial Wire** y aparece marcado con las letras **SWO** (Serial Wire Output). Pero nosotros no necesitamos emplear ese modo de depuración y lo que vamos a hacer es resetearlo para poder emplearlo como pin de control del movimiento del motor (ver tabla ??) más adelante. Para resetearlo lo seleccionamos con el ratón el esquema mostrado en el panel *pinout view* y en el menú desplegable que aparece seleccionamos **Reset State**. El pin se pondrá de color gris.

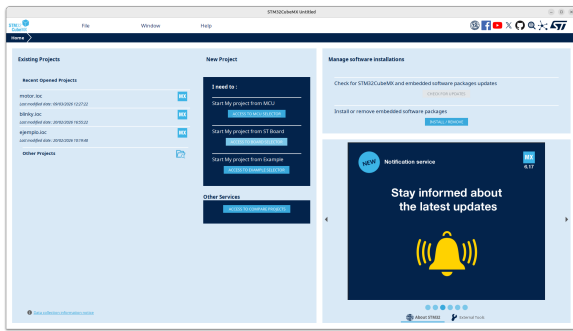
Vamos a establecer nuestras necesidades, de acuerdo con las posibilidades de la placa.

3.2. Configuración del Reloj.

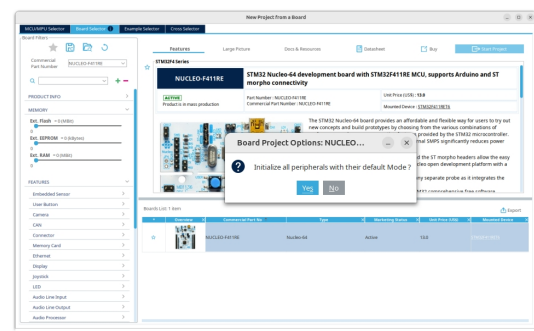
Una de las características interesantes de la *Black Pill* es su velocidad. Vamos a configurar el reloj de la placa para que trabaje a 10MHz, que es la máxima velocidad disponible.

Por defecto, la placa activa un reloj interno. Sin embargo, es mejor emplear el oscilador externo para generar la señal de reloj, ya que nos da más precisión y nos va a permitir manejar con más velocidad y precisión las comunicaciones a través del puerto serie.

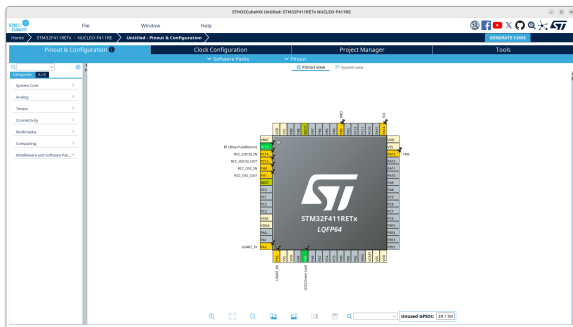
Para ello, volvemos a la categoría **System Core** y seleccionamos la entrada **RCC** (Reset and Clock Control). Buscamos la opción **High Speed Clock (HSE)**. Cambiamos "Disable" por **BYPASS Clock Source**. Hacemos esta elección porque en la placa, la señal de reloj la toma el microcontrolador directamente del programador (ST-Link). Los pines PH0 y PH1 aparecen ahora de color verde.



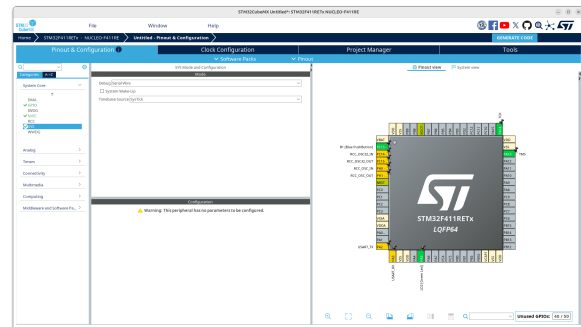
(a) Ventana inicial de STM32CubeMX



(b) Selección de la placa



(c) Layout de la placa NUCLEO F411RE



(d) Debug a Serial wire y Reseteo del pin PB3

Figura 3.1: Ventanas de inicio y selección de placa

Configurar la frecuencia en el *clock Tree* Buscamos ahora, en las pestañas de la parte superior, la que aparece marcada como *Clock Configuration*. Se nos muestra un esquemático del árbol de configuración del reloj del microprocesador.

- Nos aseguramos de que la casilla **Input Frequency** aparece el número 8 (8MHz, corresponde a la frecuencia de la señal de reloj envía al microcontrolador desde fuera).
- En el multiplexor **PLL Source Mux**, marcamos la opción **HSE**.
- En la casilla **HCLK (MHz)**, escribimos 100 (que es la velocidad máxima de la F411).
- Pulsamos **Enter** y dejamos que CubeMX ajuste los divisores M,N,P,Q.

La figura ?? muestra una imagen de la pestaña *Clock Configuration*. Con el reloj ajustado a 100mHz.

3.3. Configuración de un puerto serie (UART), a través del puerto USB de la placa.

Para poder interactuar con el motor desde un ordenador, la que vamos a hacer es comunicarnos con la placa a través de un puerto serie.

El conector USB de la placa Núcleo F411RE que usaremos para programarla y darle energía ya tiene un conversor de puerto serie integrado (a través del chip ST-LINK que está en la parte superior de la placa). Físicamente, este chip está conectado a los pines PA2 (Transmisor - TX) y PA3 (Receptor - RX) de tu procesador, que corresponden al periférico USART2.

Para configurarlo, en la ventana del STM32CubeMX, volvemos a seleccionar la pestaña de la izquierda *Pinout & Configuration*. En el menú de la izquierda, desplegamos la categoría *Connectivity* y hacemos clic en USART2.

En la ventana central (Mode), cambiamos *Disable* por *Asynchronous*. Los pines PA2 y PA3 cambian ahora a verde

Abajo, en la pestaña *Parameter Settings*, la velocidad (Baud Rate) por defecto es 115200 Bits/s. Lo dejamos así, es la velocidad estándar. Simplemente habrá que tener en cuenta ese dato en la aplicación en Python que emplearemos en el ordenador para comunicarnos con la placa.

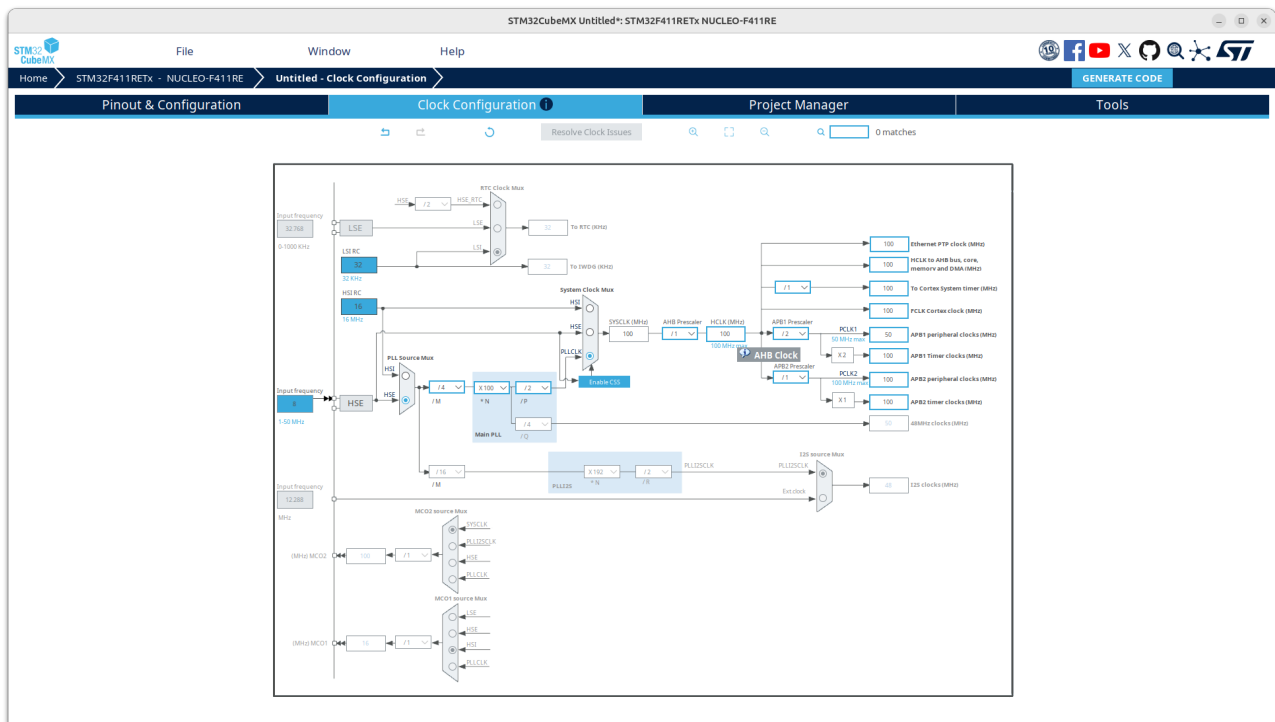


Figura 3.2: Vista de la pestaña de configuración del reloj, con el reloj configurado a 100MHz.

Por último vamos a habilitar una interrupción que permita a la placa recibir comandos desde el ordenador. Para ello, bajamos al panel **Configuration**, seleccionamos la pestaña **NVIC Settings** y marcamos la casilla **USART2 global interrupt**

3.4. Configuración del resto de pines para el control del motor

Reproducimos a continuación la tabla del capítulo 2 del guión de la práctica, en que se describen los pines que se emplearán para suministrar potencia al motor y leer las señales de los encoders.

Componente	Señal Motor/Driver	Pin STM32	Función del Periférico (STM32CubeMX)
Encoder	Canal A (Fase A)	PA0	TIM2_CH1 (Modo Encoder)
	Canal B (Fase B)	PA1	TIM2_CH2 (Modo Encoder)
Driver Potencia	PWM (Voltaje)	PB4	TIM3_CH1 (Generación PWM)
	DIR (Sentido)	PB3	GPIO_Output (Salida Digital)
	Habilitar p. H ENA	PA10	GPIO_Output (Salida Digital)
	Habilitar p. H ENB	PA7	GPIO_Output (Salida Digital)

Tabla 3.1: Mapa de conexiones nativas entre el sistema electromecánico y la placa STM32 Nucleo.

Como se deduce de la tabla, la interacción con la planta se delega en gran medida al hardware:

- **Lectura del Encoder (PA0 y PA1):** A diferencia de sistemas más simples donde se programan interrupciones por software para contar los pulsos, aquí conectamos las señales en cuadratura a los canales 1 y 2 del **Timer 2**. Al configurarlo en *Encoder Mode*, el hardware del STM32 detecta automáticamente el sentido de giro y actualiza el registro del contador (**CNT**). Nuestro software simplemente leerá este registro cuando lo necesite.

Para configurarlo adecuadamente, en el panel lateral izquierdo **Categories** de la pestaña **pinout & configuration**, Seleccionamos la entrada **timers**. Elegimos el timer **TIM2**. Se abrirá un nuevo panel para configurarlo. En dicho panel buscamos la opción **Combined Channels**, desplegamos el menú y seleccionamos **Encoder Mode** Automáticamente nos pondrá los pines **PA0** y **PA1** en color verde y les asignará, respectivamente los canales **CH1** y **CH2** del timer 2.

- **Generación de PWM (PB4):** Se conecta al canal 1 del **Timer 3**. El microcontrolador generará la señal de control de potencia (*Duty Cycle*) en este pin basándose en el valor que calculemos en nuestro bucle de control y escribamos en su registro de comparación (**CCR1**). Para configurarlo,

1. En la vista del microcontrolador (el dibujo del chip), buscamos el pin PB4, hacemos clic sobre él y seleccionamos TIM3_CH1. El pin se pondrá amarillo.
 2. En la lista de la izquierda, seleccionamos Timers ->TIM3.
 3. En Clock Source, seleccionamos Internal Clock.
 4. En Channel 1, seleccionamos PWM Generation CH1.
 5. Abajo, en Parameter Settings, ponemos
 Prescaler: 0
 Counter Period (ARR): 1000 - 1
- **Señal de Dirección (PB3):** Se configura como una salida digital estándar (GPIO Output). Escribir un nivel lógico alto (1) hará girar el motor en un sentido, y un nivel bajo (0) en el sentido opuesto, invirtiendo la polaridad en el puente en H. Para configurarlo, es suficiente con hacer click sobre el pin y seleccionar GPIO_output.
 - **Habilitar ambos puentes en H (PA10, PA7)** Habilitamos los dos puentes en H para alimentar el motor a través de ambos puentes a la vez. Estos pines deben estar siempre a nivel lógico alto (1). Los configuramos de modo análogo al anterior: hacemos click sobre ellos y seleccionamos GPIO_output.
 - **Emplear el botón azul de usuario para invertir manualmente el sentido de giro.** Realmente, esto no es necesario salvo para controlar el manual el sentido de giro. El botón azul de la placa está conectado al pin C13. Lo que vamos a hacer es asociarlo a una interrupción para que cada vez que se pulse, la placa cambie el valor del pin inmediatamente. Para ello, Hacemos click en el pin C13 y seleccionamos GPIO_EXTI13. Para activar la interrupción,
 1. En la lista de la izquierda, vamos a System Core ->NVIC.
 2. Buscamos la línea que dice EXTI line[15:10] interrupts y marcamos la casilla Enabled.

3.5. Habilitar un timer para que lea periódicamente el número de pasos (Encoders) que ha avanzado el motor.

La lectura de encoders se lleva a cabo a muy alta velocidad. Esto dará robustez a los controladores que desarrollaremos en la práctica, pero no es posible enviar a ese ritmo datos a través del puerto serie. Vamos a habilitar un tercer timer (TIM4) para que, genere una interrupción a tiempo fijo y envíe datos al ordenador por el puerto serie. Para configurarlo,

1. Volvemos de nuevo al panel de la izquierda y en Timers seleccionamos TIM4.
2. En el panel que se despliega inmediatamente a su derecha marcamos la casilla Internal Clock
3. Elegimos de momento Enviar un mensaje cada 0,01s, es decir vamos emplear para los envíos una frecuencia de 100Hz. Para ajustar la frecuencia, empleamos la expresión

$$F_{\text{interrupción}} = \frac{F_{\text{reloj}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}.$$

Hemos ajustado el reloj de nuestro microcontrolador a 100MHz. Así que si ajustamos el PSC (Prescaler) a 1000 - 1, estaremos dividiendo la frecuencia del reloj entre mil. Si además ajustamos el ARR (Counter period) también a 1000 - 1, volvemos a dividir la frecuencia del reloj entre mil, de modo que la frecuencia de la interrupción quedará $1e^8/1e^3/1e^3 = 100Hz$.

Bajamos configuration y en la pestaña Parameter Settings. Buscamos la sección Counter settings. EN Prescaler escribimos 1000 - 1 y En Counter Period la misma cantidad 1000 - 1.

4. Por último, para activar la interrupción, en la pestaña NVIC Settings Marcamos la casilla TIM4 Global interrupt.

3.6. Instalar y activar CMSIS-DSP

Por último, antes de generar nuestro proyecto, vamos a configurarlo para que use la librería CMSIS-DSP (Cortex Microcontroller Software Interface Standard - Digital Signal Processing).

Es una suite de librerías desarrollada directamente por ARM (los creadores de la arquitectura de tu STM32) y está diseñada específicamente para exprimir al máximo los procesadores Cortex-M.

Incluye un amplio número de funciones: Basic math functions, Fast math functions, Complex math functions, Filtering functions, Matrix functions, Transform functions, Motor control functions, Statistical functions, Support functions, Interpolation functions, Support Vector Machine functions (SVM), Bayes classifier functions, Distance functions, Quaternion functions.

En CMSIS-DSP, las rutinas están escritas usando instrucciones en ensamblador específicas de ARM (como multiplicaciones y acumulaciones en un solo ciclo de reloj - MAC) y usa la FPU (Unidad de Coma Flotante) por hardware.

En nuestro caso, las vamos a emplear para implementar un PID y para realizar operaciones matriciales que nos permitan emplear control por realimentación en variables de estado estimadas.

Para instalar las librerías CMSIS-DSP, seleccionamos en la barra de menú superior de la pestaña **Pinout & configuration**, la opción que Software Packs y hacemos clic en **Select Components**. Se abrirá una ventana nueva con una lista de librerías. (Nota: Si es la primera vez que se abre, puede que tarde unos segundos en conectarse a los servidores de ST para descargar el catálogo).

1. Desplegamos la pestaña **ARM.CMSIS**. Buscamos la fila que dice **CMSISDSP**.
2. En la columna de la derecha (llamada Selection), hacemos clic y cambiamos la opción a **Source**.
3. Hacemos clic en OK para cerrar esa ventana.
4. Volvemos a la pantalla principal del CubeMX. En la columna de la izquierda, abajo del todo seleccionamos **Software Packs**.
5. La desplegamos y hacemos clic en **ARM.CMSIS**.
6. En el panel central, aparecerá la opción **CMSIS DSP**. Activamos su casilla.

3.7. Definir CMake para la compilación y el enlazado y obtener el esqueleto del código del proyecto

Por último, dado que vamos a utilizar las herramientas de Visualstudio Code para compilar nuestro código, debemos añadir a nuestro proyecto las instrucciones para hacerlo empleando **CMAKE**, ya que esta es en Visualstudio la opción por defecto.

Para ello, seleccionamos la pestaña **Project Manager**. en la sección **Project Name** damos un nombre al proyecto, por ejemplo "motor". En la sección **Toolchain / IDE**, cambiamos la opción a **CMake**.

Con esto, hemos completado la configuración de nuestro proyecto. Solo queda pulsar el botón **GENERATE CODE**, situado arriba a la derecha de la ventana principal de CubeMX, para que se generen los archivos del lo que podríamos llamar el esqueleto de nuestro código. Creará una nueva carpeta con el nombre del proyecto (motor) y varias subcarpetas con todo el código generado. En la siguiente sección examinaremos en más detalle este código.

Toda la información de configuración del proyecto queda guardada en un archivo con el mismo nombre del proyecto y extensión **.ioc** (**motor.ioc**). Podemos modificar la configuración de nuestro proyecto, abriendo dicho fichero con el CubeMX. Recuperaremos todas las opciones que hemos ido seleccionando a lo largo de este capítulo y tendremos la opción de cambiarlas o añadir nuevas opciones. Cada vez que completemos una nueva configuración deberemos pulsar la opción **GENERATE CODE** para generar un nuevo fichero **.ioc** con los cambios.

A partir de ahora, para simplificar las explicaciones, nos referiremos siempre a nuestro proyecto y al directorio que contiene su código con el nombre de *motor*.

Capítulo 4

Programación del Microcontrolador en C

Una vez generado el proyecto con STM32CubeMX. Tenemos la estructura básica para controlar nuestro motor, pero ahora es necesario escribir el código. Para hacerlo, abrimos el directorio (open folder) motor en Visual Studio Code.

4.1. Estructura del proyecto

La carpeta motor está configurada siguiendo al estructura que se muestra en el siguiente árbol de directorios:

```
motor/
├── .settings/
├── .vscode/
├── build/
├── cmake/
├── Core/
│   ├── Inc/
│   │   ├── main.h
│   │   ├── motor.h
│   │   ├── stm32f4xx_hal_conf.h
│   │   └── stm32f4xx_it.h
│   └── Src/
│       ├── main.c
│       ├── motor.c
│       ├── stm32f4xx_hal_msp.c
│       ├── stm32f4xx_it.c
│       ├── syscalls.c
│       ├── system.c
│       └── system_stm32f4xx.c
├── Drivers/
├── .clangd
├── .gitignore
├── .mxproject
├── .project
├── CMakeLists.txt
├── CMakePresets.json
├── motor.ioc
├── Motor.py
├── registro_motor_20260506_171311.csv
├── startup_stm32f411xe.s
└── STM32F411XX_FLASH.ld
```

Todos los directorios y ficheros que comienzan por un punto, contienen archivos de configuración del proyecto, creados automáticamente por Visual Studio, o Git. También lo son los que aparecen en el directorio /motor creados por CMake, (CMakeLists.txt, CMakePresets.json) y el propio directorio cmake/. O los relacionados con la descarga de código a la placa, (startup_stm32f411xe.s, STM32F411XX_FLASH.ld) creados una vez se compila y descarga el código.

El directorio `build/` se genera al compilar el proyecto. De hecho, no aparece hasta que se compila por primera vez y podría borrarse antes de volver a compilar y CMake lo generaría de nuevo.

El directorio `Drivers/`, contiene las rutinas de CMSIS que emplearemos para desarrollar el código de control del motor y las rutinas de la *HAL* (Hardware (Hardware Abstraction Layer o Capa de Abstracción de Hardware) en microcontroladores es un conjunto de funciones de software que actúa como puente entre el código de aplicación y los registros físicos del hardware. Permite programar periféricos (GPIO, I2C, SPI) con comandos genéricos.

Hemos dejado casi para el final, El directorio principal de nuestro proyecto `Core/`. Este directorio contiene a su vez, otros dos `Inc/` y `Src/`. El primero contiene los archivos de cabecera (.h) y el segundo los archivos fuente en c (.c). Todos estos archivos han sido generados automáticamente por STM32CubeMX. Los archivos `main` y `motor`, ofrecen simples esqueletos en los deberemos añadir el código necesario para programar nuestra placa. Vamos a verlos en detalle en las proximas secciones.

4.2. `main.h` y `main.c`

Vamos a empezar por los archivos principales. El archivo de cabecera, `main.h`, es un archivo breve. Lo vamos a usar tal cual lo genera STM32CubeMX.

4.3. Configuración de Periféricos (STM32CubeMX)

Para interactuar con la planta (el motor de continua), es necesario configurar los periféricos de hardware del microcontrolador. Esta tarea se realiza mediante la herramienta gráfica *STM32CubeMX*, la cual genera el código de inicialización (*drivers HAL*). Los elementos fundamentales a configurar son:

- **Lectura del Encoder (TIM2):** Se configura un temporizador en modo *Encoder Mode*. El hardware del STM32 se encargará automáticamente de decodificar las señales en cuadratura (Canal A y Canal B) y actualizar un registro contador (CNT), liberando a la CPU de esta tarea.
- **Generación de PWM (TIM3):** Se configura un temporizador para generar una señal PWM (Modulación por Ancho de Pulso) a una frecuencia superior a 10 kHz para evitar ruido audible y asegurar una dinámica fluida en el puente en H.
- **Bucle de Control (TIM4):** Un temporizador configurado para generar una interrupción periódica exacta cada 10 ms ($T_s = 0,01$ s). Esta interrupción garantiza que nuestro bucle de control se ejecute de forma determinista.
- **Comunicaciones (USART2):** Se habilita el puerto serie asíncrono configurado a 115200 baudios y con interrupciones de recepción (RX) habilitadas, para recibir los comandos desde nuestro *Dashboard* en Python.